Parallel Simulation of Microplastic Dispersion Using Ocean Current Data: Methodology

Patil Rigved Ramesh
Dept. of Information Technology
National Institute of Technology Karnataka, Surathkal
rigved.241it052@nitk.edu.in

Batti Akbarali Abbasali
Dept. of Information Technology
National Institute of Technology Karnataka, Surathkal
akbar.241it014@nitk.edu.in

Dhruv Agrawal
Dept. of Information Technology
National Institute of Technology Karnataka, Surathkal
dhruvagrawal.241it026@nitk.edu.in

Salim Moh
Dept. of Information Technology
National Institute of Technology Karnataka, Surathkal
salimmoh.241it068@nitk.edu.in

***Abstract*—We lay out the methodology we plan to use to evaluate an OpenMP-parallelized version of Lagrangian particle advection for simulating microplastic dispersion from ocean current data. Each particle is a passive tracer: its velocity comes from bilinearly interpolating the surrounding current grid and linearly interpolating between the two nearest time slices, and its position is advanced with a fixed-step Runge–Kutta integrator. Since particles never interact, the advection loop is naturally parallel across the particle array, and we parallelize it with OpenMP, paying attention to scheduling policy and data layout. Below we describe the current-field data we use, the method itself, and the experimental plan we will follow to compare against a serial baseline.**

***Index Terms*—Lagrangian Particle Tracking, Microplastic Dispersion, OpenMP, Ocean Current Interpolation, Runge–Kutta Integration, Parallel Scalability**

I. Methodology

Microplastic pollution has spread across every ocean basin, and studying its transport at a particle count and resolution useful for environmental analysis is computationally expensive under serial execution. This report describes the methodology we use to design, implement, and evaluate a parallel (OpenMP-based) Lagrangian advection framework against a matched serial baseline, using real and synthetic ocean current data.

We organize the methodology around four research questions that the experiments are designed to answer.

- **RQ1:** How much speedup and parallel efficiency does the OpenMP version give over a serial baseline, and how does that change with particle count and thread count?
- **RQ2:** Is the advection kernel limited by compute or by memory bandwidth, and does that depend on current-field resolution?
- **RQ3:** Does the choice of OpenMP scheduling policy, or how particle data is laid out in memory, meaningfully change thread load balance?
- **RQ4:** Does parallelizing the simulation change the physical result — do the dispersion patterns still match the serial run?

The rest of this section covers the datasets we use (Section I-A), the proposed method itself (Section I-B), and the experimental settings and evaluation metrics used to answer RQ1–RQ4 (Section I-C).

A. Dataset Explanation

We use two categories of current-field data so that the framework can be validated under both controlled and real-world conditions: a synthetic analytic flow for correctness checks, and real satellite/model-derived ocean current products for physically meaningful evaluation. Table I summarizes the five current fields used across our seven experimental configurations (the synthetic field is swept over three perturbation values, described below).

TABLE I
Ocean Current Datasets

Region	Type	Eddy Activity	Resolution
Double-Gyre (ϵ-swept)	Synthetic	Controlled	Analytic
N. Pacific Gyre Interior	OSCAR	Very low	0.33°, 5-day
California Current	OSCAR	Moderate	0.33°, 5-day
Gulf Stream Extension	HYCOM	High	1/12°, daily
Agulhas Retroflection	HYCOM	High	1/12°, daily

Each current field is stored as a regular latitude–longitude grid of (u, v) surface velocity values per time slice, distributed as NetCDF files and read through a custom NetCDF loader. Every run releases particles from the same fixed set of starting points within a region, so that timing and trajectory comparisons across implementations and scheduling strategies remain

fair. The rest of this subsection answers, for each data source: what it is, where it comes from, what it contains, and how it supports our research objectives.

1) Synthetic Data: Idealized Double-Gyre: **What.** We use an analytic double-gyre flow field with perturbation amplitude $\epsilon \in \{0.05, 0.15, 0.30\}$, producing low-, medium-, and high-shear cases respectively.

Where it comes from. The field is not measured but generated in-house from the standard closed-form double-gyre velocity equations (a well-known idealized model used in Lagrangian transport studies), evaluated directly in our code at any (x, y, t) — no external download is needed.

What it contains. A time-dependent, analytically-defined (u, v) velocity field over a unit square domain, together with the exact closed-form particle trajectories that the equations predict, which serve as ground truth.

How it supports our objectives. Because exact trajectories are known in closed form, this dataset lets us verify that our interpolation and RK4 integration code is correct before trusting it on real data, and lets us isolate the effect of eddy shear (ϵ) on parallel performance in a fully controlled setting (RQ2), independent of the noise and gaps present in real satellite/model products. It is also the basis for the drift check used to answer RQ4, since the serial and parallel outputs can both be compared against an exact reference.

2) Real-World Data: OSCAR and HYCOM: **What.** We validate on four ocean regions (Table I) chosen to span a wide range of eddy activity: the North Pacific Gyre Interior, the California Current, the Gulf Stream Extension, and the Agulhas Retroflection.

Where it comes from. Two real current-field products are used. *OSCAR* (Ocean Surface Current Analysis Real-time) is a satellite-derived surface current product distributed by NASA's Physical Oceanography Distributed Active Archive Center (PO.DAAC), combining sea-surface-height altimetry, scatterometer winds, and drifter data. *HYCOM* (HYbrid Coordinate Ocean Model) is a data-assimilative ocean reanalysis produced by the HYCOM Consortium (led by the U.S. Naval Research Laboratory) and distributed through the HYCOM data server. Both are downloaded as NetCDF files for the specific regions and time windows listed in Table I.

What it contains. Each file provides gridded zonal and meridional surface velocity components (u, v, in m/s) indexed by latitude, longitude, and time: OSCAR at 0.33° spatial resolution with 5-day temporal spacing, and HYCOM at 1/12° spatial resolution with daily spacing (Table I). We additionally use a land/coastline mask for the boundary-handling step (Step 5, Section I-B).

How it supports our objectives. The four regions are chosen specifically to span calm to highly turbulent, coarse to fine-resolution conditions:

- **North Pacific Gyre Interior** — calm, coarse-resolution water, used as a negative control; even a simple serial update should handle this case easily, and it gives the earliest expected onset of memory-bandwidth limits (RQ2).
- **California Current** — an intermediate case with moderate eddy activity, useful for interpolating between the calm and turbulent extremes.
- **Gulf Stream Extension** and **Agulhas Retroflection** — fast, fine-resolution, eddy-rich regions that resemble the real convergence zones where floating plastic accumulates. These are the regions we care most about getting right, both physically (RQ4, since they are the target use case described above) and computationally, since coastline proximity and dense eddies stress the load-balance question in RQ3.

Together, these four regions let us test whether speedup, efficiency, and dispersion accuracy hold not just in an idealized setting but across the realistic range of resolution and eddy activity a marine researcher or coastal agency would actually encounter.

B. Proposed Method Explanation

1) Overview: The simulation advances in fixed timesteps. At every step t_k, each particle's velocity is reconstructed from the current field and its position is advanced before moving on to t_{k+1}. Because particles do not interact with one another within a timestep, updating one particle never depends on another particle's state — this independence is precisely what makes the per-timestep loop well suited to parallelization. The only real design decisions are (i) how the particle array is split across threads, and (ii) how the current-field data is arranged in memory to keep those accesses cache-friendly.

Fig. 1 shows the block diagram of one pass of this loop.

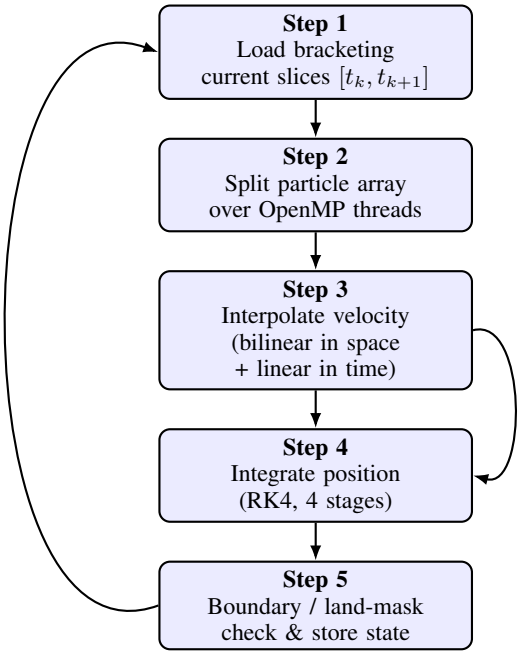


Fig. 1. Per-timestep parallel advection loop.

Step 1 selects the two current-field time slices that bracket t_k, so temporal interpolation is well defined. Step 2 hands

each OpenMP thread a chunk of the particle array; because the current-field grid is read-only, threads never need to synchronize while updating particles. Step 3 looks up the velocity at each particle's location using bilinear interpolation in space and linear interpolation in time. Step 4 uses that velocity to advance the particle with a fixed-step 4th-order Runge–Kutta scheme, calling back into Step 3 at each of the four stages. Step 5 applies a land-mask/boundary rule and records the new position, and the loop repeats until the simulation reaches its end time.

2) Baseline: Serial Advection: The baseline is a plain single-threaded loop that runs Steps 3–5 for every particle, using exactly the same interpolation and integration code as the parallel version. It has no coordination overhead at all, but its runtime grows linearly with particle count and cannot use more than one core. We use it both to check correctness and as the timing reference for measuring speedup.

3) Proposed Strategy: Parallel Particle Advection: We wrap Steps 3–4 in an `omp parallel for` region. Work per particle is not perfectly uniform: particles near a coastline incur extra land-mask checks, and particles near a grid-cell edge cost an extra memory fetch. Rather than assume one scheduling policy is best a priori, we compare three:

- **Static** scheduling has the lowest dispatch overhead but can leave threads unbalanced if slow particles cluster together.
- **Dynamic** scheduling rebalances work at runtime, at the cost of a shared work queue.
- **Guided** scheduling starts with large chunks and shrinks them near the end, aiming for a middle ground between the two.

Algorithm 1 summarizes the update; which policy performs best is tested empirically under RQ3 rather than assumed in advance.

We also store particle state as separate contiguous arrays for longitude, latitude, and trajectory history (structure-of-arrays, SoA) rather than as an array of structs (AoS), so that a thread's memory accesses stay stride-friendly. The current-field grid is stored the same way, and we keep the two bracketing time slices resident in memory so that Step 3's lookups tend to stay in cache.

TABLE II
RUNTIME SOFTWARE STACK

Component	Implementation
Language	C++17
Parallelism	OpenMP thread pool
Particle layout	Structure-of-Arrays
Interpolation	Bilinear (space) + linear (time)
Integrator	Fixed-step 4th-order Runge–Kutta
Current-field I/O	Custom NetCDF loader

4) Complexity: Each particle-timestep performs a fixed amount of work: 4 corner reads $\times$ 2 time slices $\times$ 4 RK4 stages, all $O(1)$ regardless of grid size. Hence, for P particles over T timesteps, total work is $O(P \cdot T)$, split roughly

Algorithm 1 Parallel Per-Timestep Advection

Require: particle array P, current slices C_k, C_{k+1}, schedule S

```
#pragma omp parallel for schedule(S)
for i ← 0 to |P| − 1 do
    for stage ← 1 to 4 do                          ▷ RK4 stages
        v ← Bilinear(C_k, C_{k+1}, P_i.pos, t)
        accumulate RK4 stage using v
    end for
    P_i.pos ← RK4-updated position
    if P_i.pos crosses land / boundary then
        apply freeze-on-land rule
    end if
    record P_i.pos
end for
t ← t + Δt
```

$O(P \cdot T/N_{\text{threads}})$ per thread. Nothing in the algorithm scales worse than linearly, so any limit on scalability observed experimentally should stem from memory bandwidth and cache behavior rather than from the algorithm itself.

C. Experimental Settings and Analysis

1) Platform and Design: All experiments run on a single multi-core CPU node, using OpenMP for the thread pool, with code compiled in C++17 using `-O3 -march=native`. A small custom NetCDF loader (Table II) is used so that timing overhead can be attributed to the advection kernel rather than to an external simulation framework.

The *parallel* implementation is run on all five current fields (seven configurations, counting the three double-gyre perturbation values), sweeping particle count $P \in \{10^4, 10^5, 10^6, 5 \times 10^6\}$, thread count $N \in \{1, 2, 4, 8, 16\}$, and the three OpenMP scheduling policies (static, dynamic, guided). The *serial* baseline has no thread count and no scheduling clause, so it is not swept over N or S: for each current field it is run exactly once per particle count P, giving the single-threaded reference time $T_{\text{serial}}(P)$ used in the speedup and efficiency computations below.

For each configuration (serial or parallel) we perform 2 warm-up runs followed by 5 timed runs. Every run within a region uses the same particle release points and simulated time span, the CPU governor is set to performance mode, and threads are pinned to reduce timing noise.

We log, per run: total wall-clock time, per-thread busy time, cache-miss rate and memory bandwidth (via `perf stat`), particle throughput, and position drift relative to the serial baseline. All measurements are buffered in memory and written to disk only after the run finishes, so that logging itself does not distort the timing.

2) Evaluation Metrics: **RQ1 — Speedup and Efficiency:** We take the median wall-clock time over 5 runs and compute

$$S(N) = \frac{T_{\text{serial}}}{T_{\text{parallel}}(N)}, \qquad E(N) = \frac{S(N)}{N} \tag{1}$$

against thread count N, for each particle count.

RQ2 — Compute vs. Memory-Bound Behavior: We estimate the kernel's arithmetic intensity and compare it against the platform's roofline model, cross-checked against measured cache-miss rate and bandwidth utilization across the three grid resolutions in Table I.

RQ3 — Scheduling and Layout Sensitivity: We compare per-thread busy-time variance across the three scheduling policies and the two data layouts (SoA vs. AoS), at fixed particle and thread count, to determine whether load imbalance is a significant factor.

RQ4 — Physical Fidelity: We measure how far particle trajectories in the parallel run drift from the serial baseline, and compare the resulting accumulation-zone maps qualitatively (by visual inspection) between the two implementations.

3) Reproducibility:

- Identical particle release points are used across implementations, per region.
- CPU governor is set to performance mode; threads are pinned to cores.
- 2 warm-up runs are discarded; the median of 5 timed runs is reported.
- Logging is kept in memory during the run and flushed only at the end.

4) Expected Outcome: We expect near-linear speedup up to a moderate thread count across all regions, with parallel efficiency dropping off soonest on the coarse, low-eddy region (North Pacific Gyre Interior) — since there is less arithmetic per byte moved there, it should hit memory-bandwidth limits earliest — and holding up longer on the high-resolution, high-eddy regions (Gulf Stream Extension, Agulhas Retroflection). We also expect dynamic or guided scheduling to help specifically in coastal regions, where land-mask branching makes per-particle work uneven. If speedup plateaus earlier than expected across the board, this would itself be a useful result, pointing to memory bandwidth rather than thread count as the primary bottleneck.